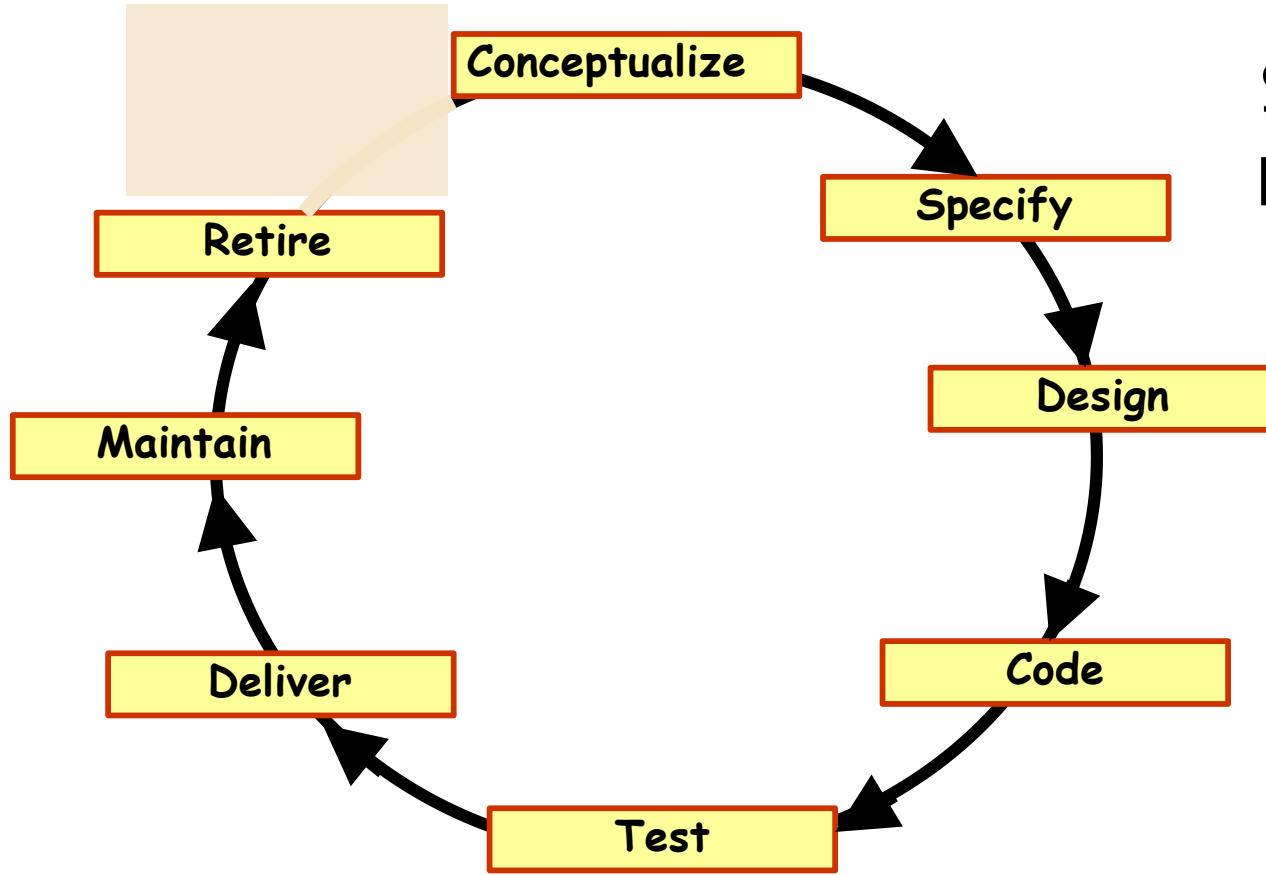




LIFE CYCLE MODELS

SOFTWARE LIFE CYCLE



LIFE CYCLE MODEL

- A software life cycle model (also process model or SDLC):
 - A descriptive and diagrammatic model of software life cycle:
 - Identifies all the activities undertaken during product development,
 - Establishes a precedence ordering among the different activities,
 - Divides life cycle into phases.

LIFE CYCLE MODEL (CONT.)

- Each life cycle phase consists of several activities.
 - For example, the design stage might consist of:
 - structured analysis
 - structured design
 - Design review

WHY MODEL LIFE CYCLE?

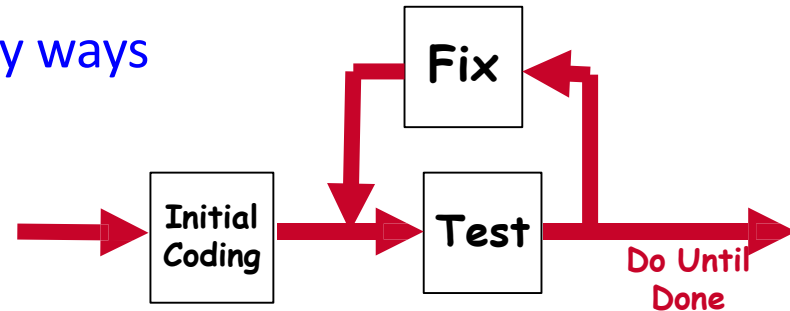
- A graphical and written description:
 - **Helps common understanding of activities among the software developers.**
 - **Helps to identify inconsistencies, redundancies, and omissions in the development process.**
 - **Helps in tailoring a process model for specific projects.**

LIFE CYCLE MODEL (CONT.)

- The development team must identify a suitable life cycle model:
 - and then adhere to it.
 - Primary advantage of adhering to a life cycle model:
 - Helps development of software in a systematic and disciplined manner.

LIFE CYCLE MODEL (CONT.)

- When a program is developed by a single programmer ---
 - The problem is within the grasp of an individual.
 - He has the freedom to decide his exact steps and still succeed --- called Exploratory model--- One can use it in many ways
 - Code → Test → Design
 - Code → Design → Test → Change Code →
 - Specify → code → Design → Test → etc.



LIFE CYCLE MODEL (CONT.)

- When software is being developed by a team:
 - There must be a precise understanding among team members as to when to do what,
 - Otherwise, it would lead to chaos and project failure.

LIFE CYCLE MODEL (CONT.)

- A software project will never succeed if:
 - one engineer starts writing code,
 - another concentrates on writing the test document first,
 - yet another engineer first defines the file structure
 - another defines the I/O for his portion first.

Phase Entry and Exit Criteria

- A life cycle model:



- defines entry and exit criteria for every phase.
- A phase is considered to be complete:
 - only when all its exit criteria are satisfied.

LIFE CYCLE MODEL (CONT.)

- What is the phase exit criteria for the software requirements specification phase?
 - Software Requirements Specification (SRS) document is complete, reviewed, and approved by the customer.
- A phase can start:
 - Only if its phase-entry criteria have been satisfied.

LIFE CYCLE MODEL: MILESTONES

- Milestones help software project managers:
 - Track the progress of the project.
 - Phase entry and exit are important milestones.



Life Cycle and Project Management

- When a life cycle model is followed:
 - The project manager can at any time fairly accurately tell,
 - At which stage (e.g., design, code, test, etc.) the project is.

Project Management Without Life Cycle Model

- It becomes very difficult to track the progress of the project.
 - The project manager would have to depend on the guesses of the team members.
- This usually leads to a problem:
 - known as the **99% complete syndrome**.

PROJECT DELIVERABLES: MYTH AND REALITY

Myth:

The only deliverable for a successful project is the working program.

Reality:

Documentation of **all aspects** of software development are needed to help in operation and maintenance.

- Many life cycle models have been proposed.
- We confine our attention to only a few commonly used models.

–Waterfall
–V model,
–Evolutionary,
–Prototyping
–Spiral model,
–Agile models

Traditional models

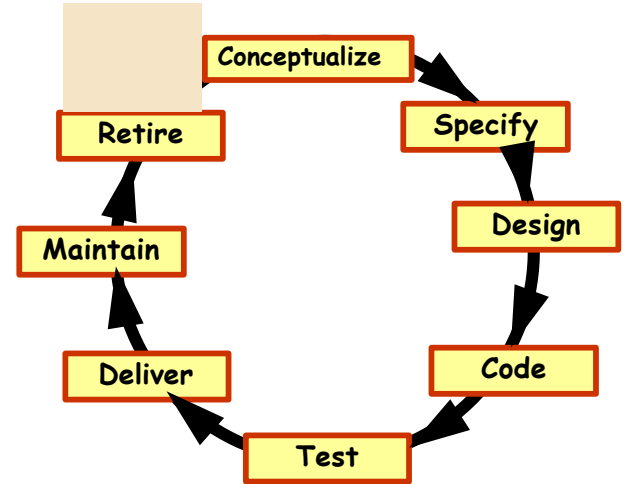
Life Cycle Model (CONT.)

- Software life cycle (or software process):
 - Series of identifiable stages that a software product undergoes during its life time:
 - Feasibility study
 - Requirements analysis and specification,
 - Design,
 - Coding,
 - Testing
 - Maintenance.

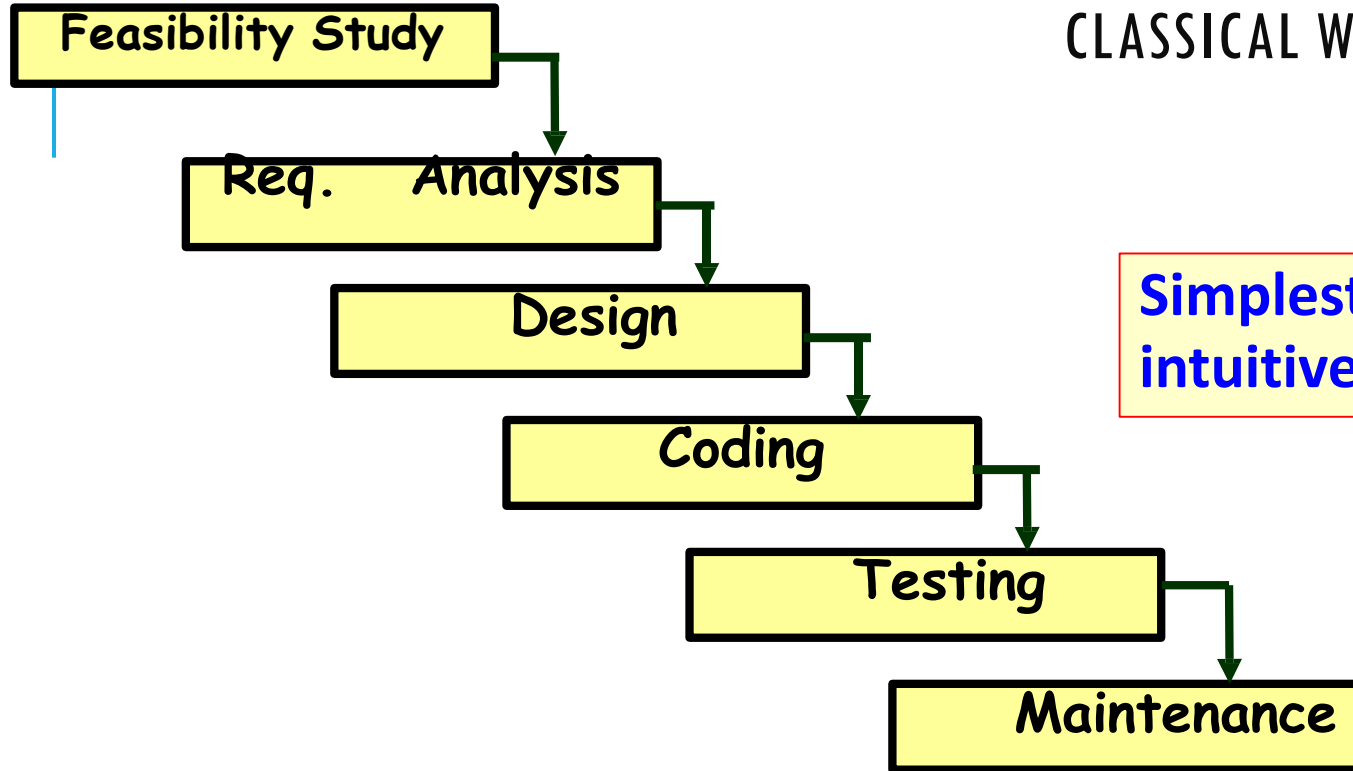
Software Life Cycle

CLASSICAL WATERFALL MODEL

- Classical waterfall model divides life cycle into following phases:
 - Feasibility study,
 - Requirements analysis and specification,
 - Design,
 - Coding and unit testing,
 - Integration and system testing,
 - Maintenance.

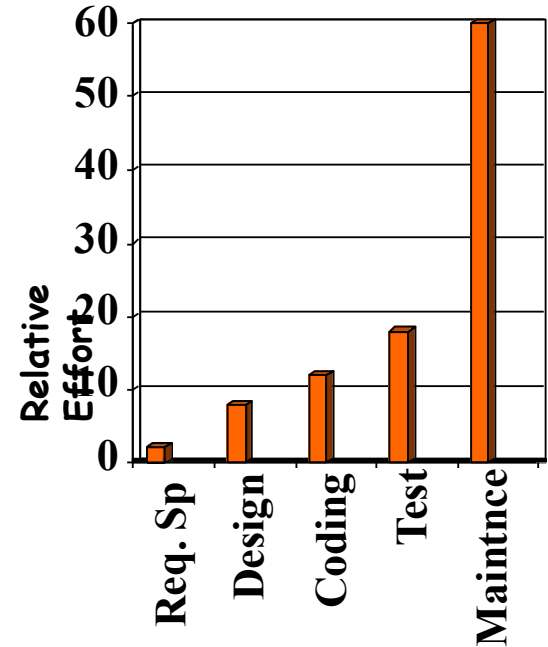


CLASSICAL WATERFALL MODEL



RELATIVE EFFORT FOR PHASES

- Phases between feasibility study and testing
 - Called **development phases**.
- Among all life cycle phases
 - **Maintenance phase consumes maximum effort.**
- Among development phases,
 - Testing phase consumes the maximum effort.



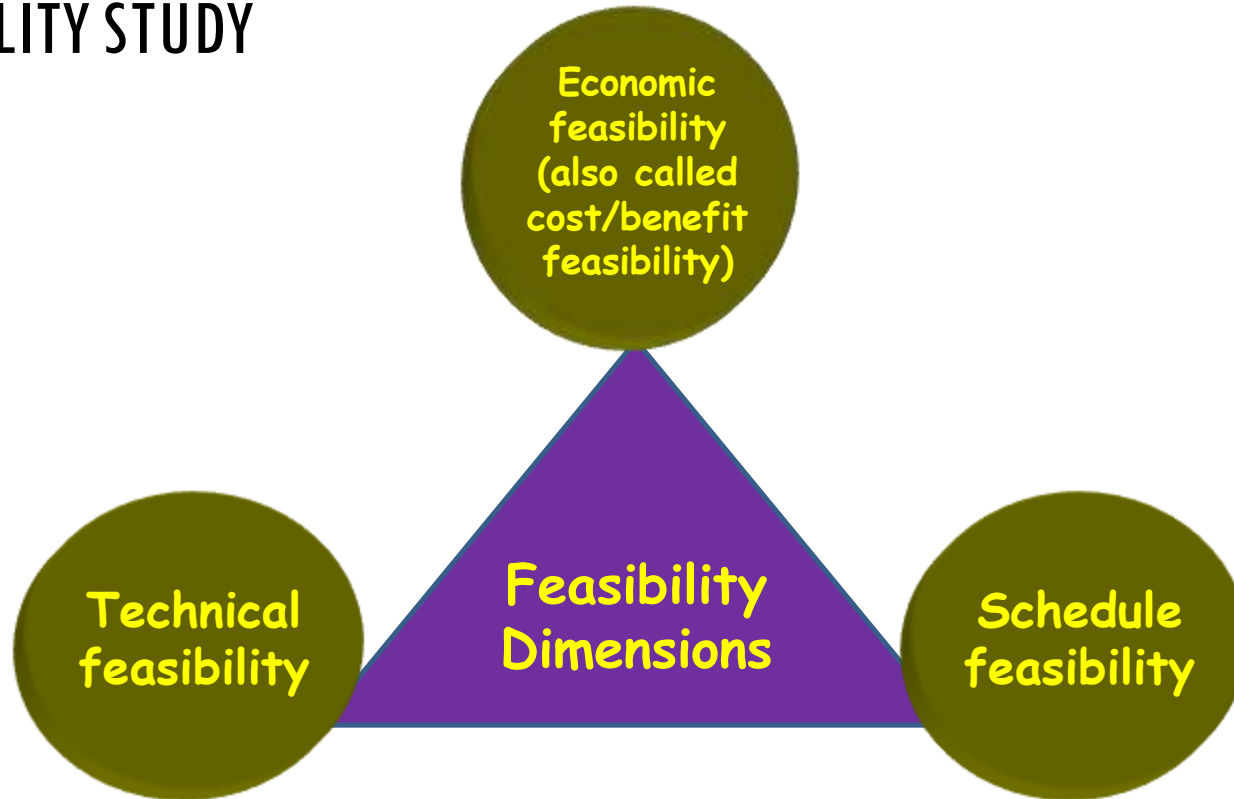
- Most organizations usually define:
 - Standards on the outputs (deliverables) produced at the end of every phase
 - Entry and exit criteria for every phase.
- They also prescribe methodologies for:
 - Specification,
 - Design,
 - Testing,
 - Project management, etc.

Process Model

CLASSICAL WATERFALL MODEL (CONT.)

- The guidelines and methodologies of an organization:
 - Called the organization's software development methodology.
- Software development organizations:
 - Expect fresh engineers to master the organization's software development methodology.

FEASIBILITY STUDY



- Main aim of feasibility study: **determine whether developing the software is:**
 - **Financially worthwhile**
 - **Technically feasible.**
- Roughly understand what customer wants:
 - **Data which would be input to the system,**
 - **Processing needed on these data,**
 - **Output data to be produced by the system,**
 - **Various constraints on the behavior of the system.**

Feasibility Study

First Step

- **SPF Scheme for CFL**
- CFL has a large number of employees, exceeding 50,000.
- Majority of these are casual labourers
- Mining being a risky profession:
 - Casualties are high
- Though there is a PF:
 - But settlement time is high
- There is a need of SPF:
 - For faster disbursement of benefits

Case Study

FEASIBILITY: CASE STUDY

- Manager visits main office, finds out the main functionalities required
- Visits mine site, finds out the data to be input
- Suggests alternate solutions
- Determines the best solution
- Presents to the CFL Officials
- Go/No-Go Decision

ACTIVITIES DURING FEASIBILITY STUDY

- Work out an overall understanding of the problem.
- Formulate different solution strategies.
- Examine alternate solution strategies in terms of:
 - resources required,
 - cost of development, and
 - development time.

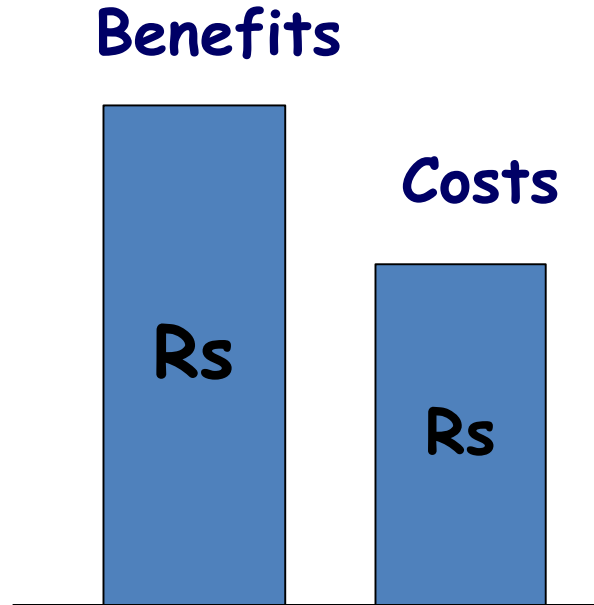
- Perform a cost/benefit analysis:
 - Determine which solution is the best.
 - May also find that none of the solutions is feasible due to:
 - high cost,
 - resource constraints,
 - technical reasons.

Activities during Feasibility Study

COST BENEFIT ANALYSIS (CBA)

- Need to identify all costs --- these could be:
 - **Development costs**
 - **Set-up**
 - **Operational costs**
- Identify the value of benefits
- Check benefits are greater than costs

THE BUSINESS CASE



- **Benefits of delivered project must outweigh costs**
- **Costs include:**
 - Development
 - Operation
- **Benefits:**
 - Quantifiable
 - Non-quantifiable

THE BUSINESS CASE

- Feasibility studies should help write a 'business case'
- Should provide a justification for starting the project
- Should show that the benefits of the project will exceed:
 - **Various costs**
- Needs to take account of business risks

WRITING AN EFFECTIVE BUSINESS CASE

1. Executive summary

2. Project background:

- The focus must be on what, exactly, the project is undertaking, and should not be confused with what might be a bigger picture.

3. Business opportunity

- What difference will it make?
- What if we don't do it?

4. Costs

- Should include the cost of development, implementation, training, change management, and operations.

5. Benefits

- Benefits usually presented in terms of revenue generation and cost reductions.

6. Risks

- Identify risks.
- Explain how these will be managed.

Feasibility Study

Req. Analysis

Design

Coding

Testing

Maintenance

CLASSICAL WATERFALL MODEL

Requirements Analysis and Specification

- Aim of this phase:
 - Understand the exact requirements of the customer,
 - Document them properly.
- Consists of two distinct activities:
 - Requirements gathering and analysis
 - Requirements specification.

REQUIREMENTS ANALYSIS AND SPECIFICATION

- Gather requirements data from the customer:
 - Analyze the collected data to understand what customer wants
- Remove requirements problems:
 - Inconsistencies
 - Anomalies
 - Incompleteness
- Organize into a Software Requirements Specification (SRS) document.

REQUIREMENTS GATHERING

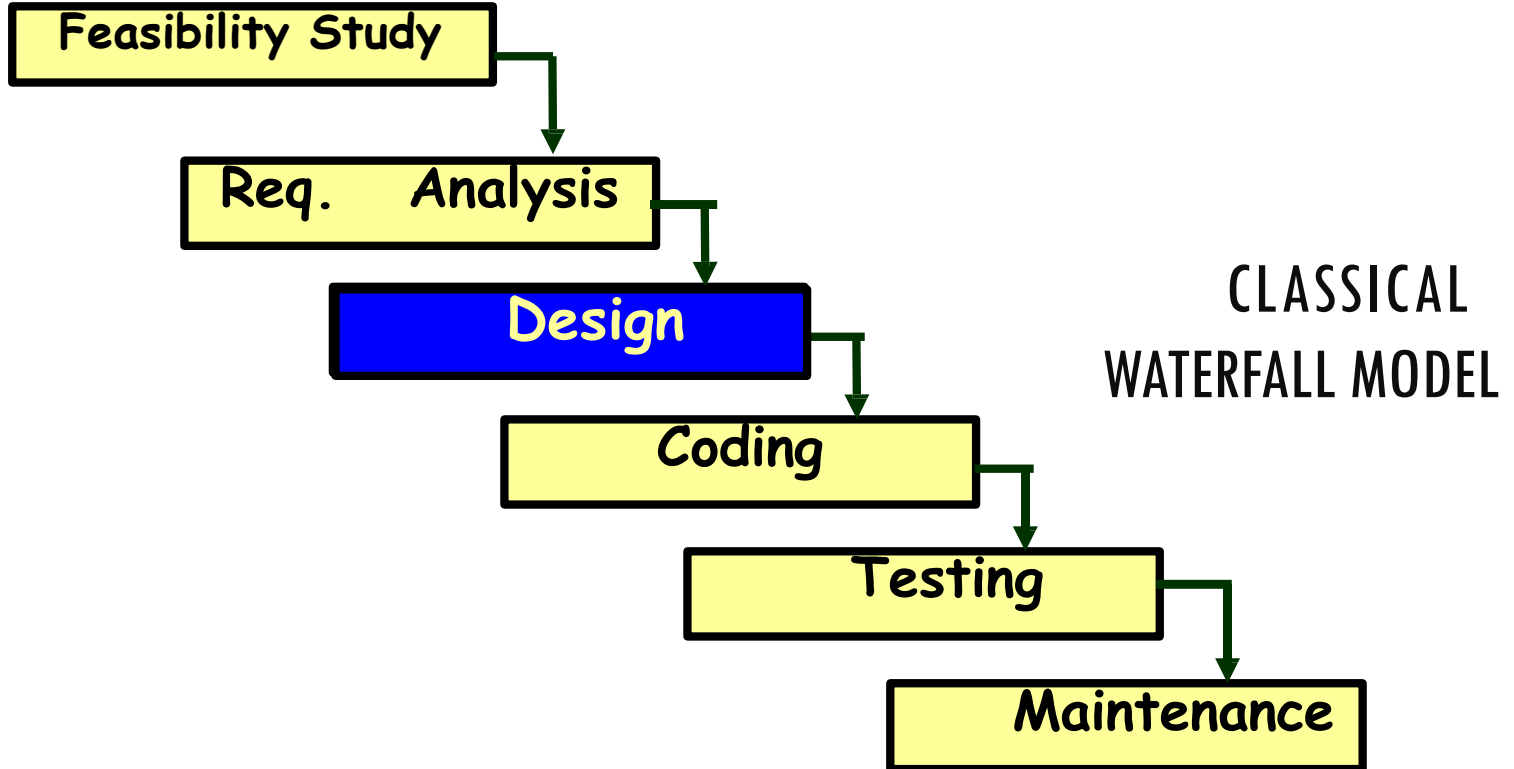
- Gathering relevant data:
 - Usually collected from the end-users through interviews and discussions.
 - **Example:** for a business accounting software:
 - Interview all the accountants of the organization to find out their requirements.

REQUIREMENTS ANALYSIS (CONT...)

- The data you initially collect from the users:
 - Usually contain several contradictions and ambiguities.
 - Why?
 - Each user typically has only a partial and incomplete view of the system.**

REQUIREMENTS ANALYSIS (CONT...)

- Ambiguities and contradictions:
 - must be identified
 - resolved by discussions with the customers.
- Next, requirements are organized:
 - into a **Software Requirements Specification (SRS)** document.



DESIGN

- During design phase requirements specification is transformed into :
 - A form suitable for implementation in some programming language.
- Two commonly used design approaches:
 - Traditional approach,
 - Object oriented approach

TRADITIONAL DESIGN APPROACH

- Consists of two activities:
 - Structured analysis (typically carried out by using DFD)
 - Structured design

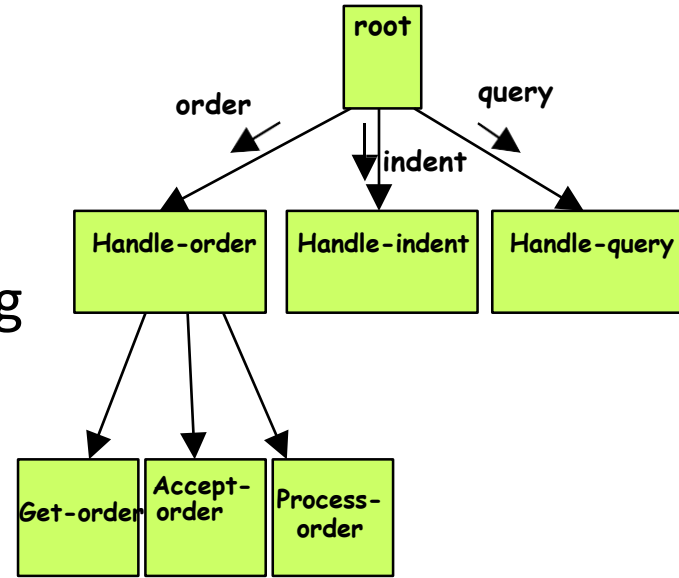
STRUCTURED DESIGN

- High-level design:

- decompose the system into **modules**,
- represent invocation relationships among the modules.

- Detailed design:

- different modules designed in greater detail:
 - data structures and algorithms for each module are designed.

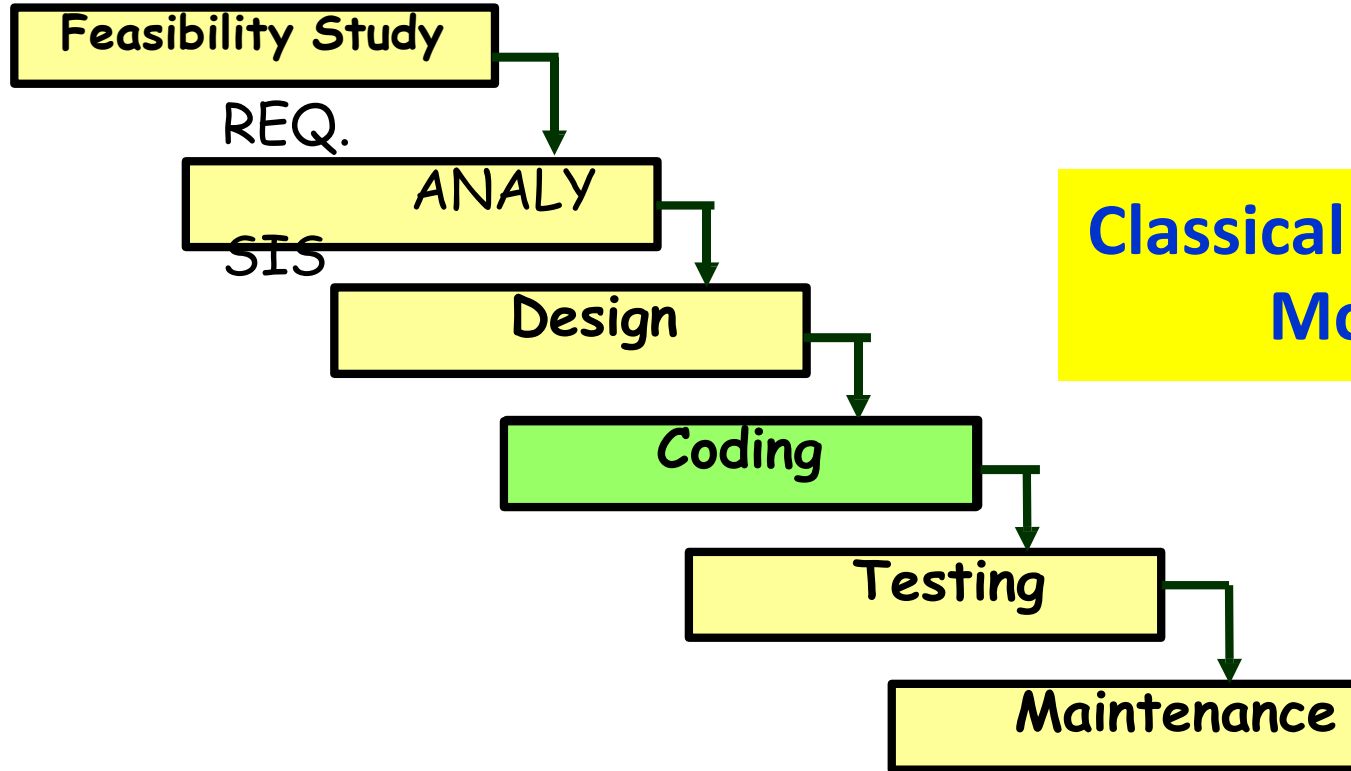


- First identify various objects (real world entities) occurring in the problem:
 - Identify the relationships among the objects.
 - For example, the objects in a pay-roll software may be:
 - employees,
 - managers,
 - pay-roll register,
 - Departments, etc.

Object-Oriented Design

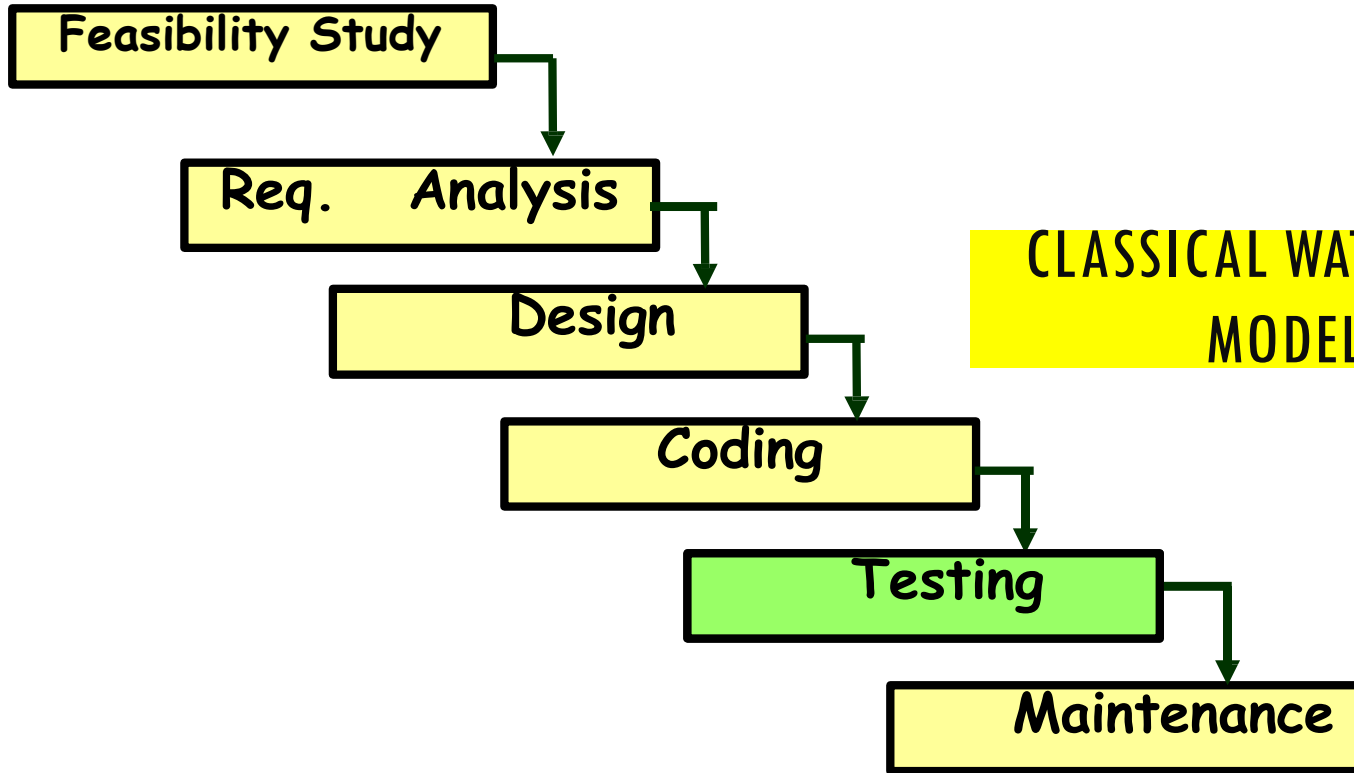
OBJECT ORIENTED DESIGN (CONT.)

- Object structure:
 - Refined to obtain the detailed design.
- OOD has several advantages:
 - Lower development effort,
 - Lower development time,
 - Better maintainability.



CODING AND UNIT TESTING

- During this phase:
 - Each module of the design is coded,
 - Each module is unit tested
 - That is, tested independently as a stand alone unit, and debugged.
 - Each module is documented.

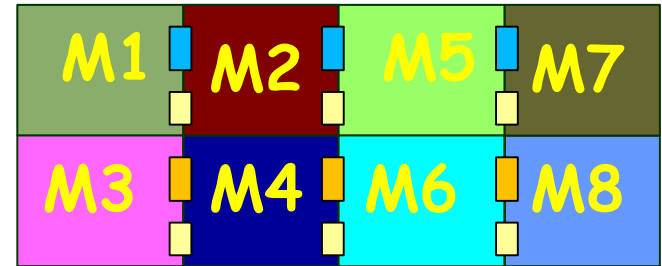


INTEGRATION AND SYSTEM TESTING

- Different modules are integrated in a planned manner:

—Modules are usually integrated through a number of steps.

- During each integration step,
 - the partially integrated system is tested.



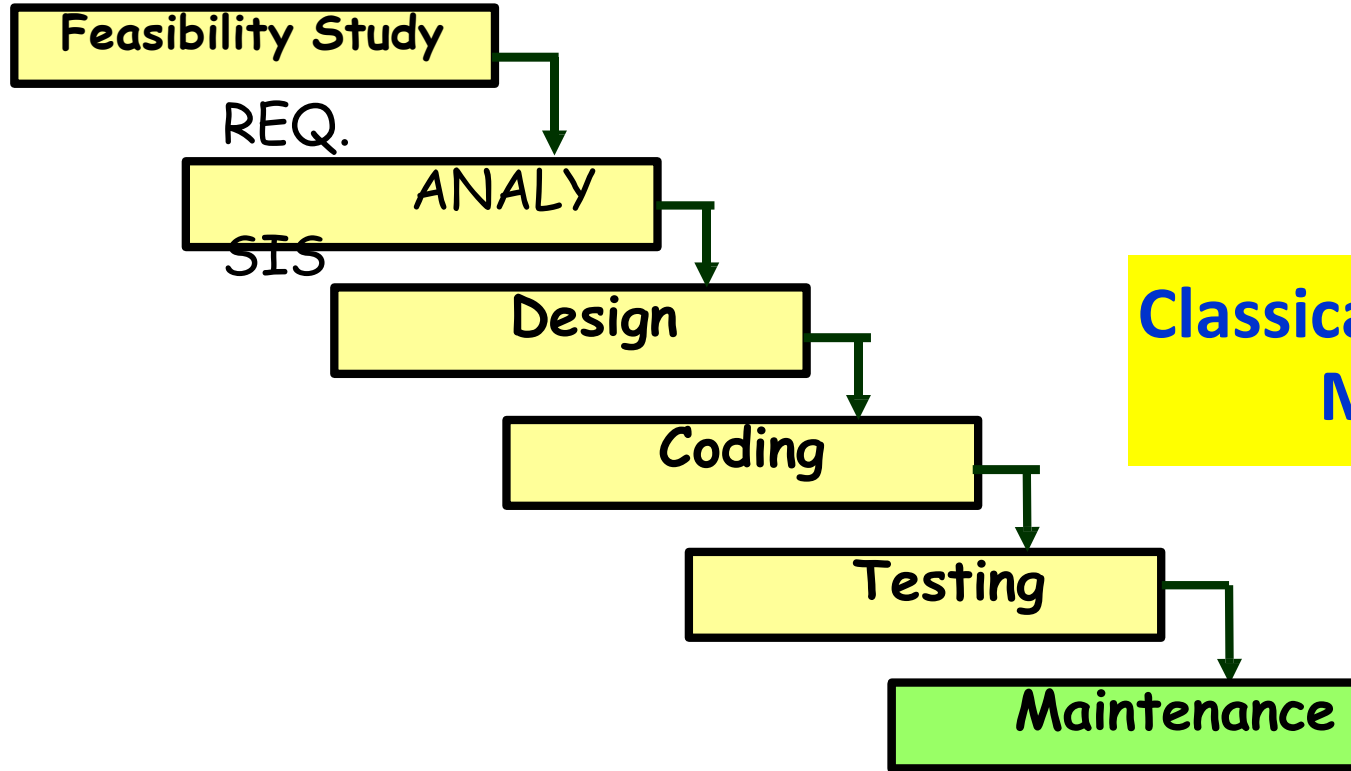
SYSTEM TESTING

- After all the modules have been successfully integrated and tested:

- System testing is carried out.

- Goal of system testing:

- Ensure that the developed system functions according to its requirements as specified in the SRS document.



Classical Waterfall Model